The Cost of Stale Telemetry in Traffic Engineering: Zero-Shot Learned Policies versus Re-Tuned Heuristics

Abstract

When a traffic-engineering controller reads stale, noisy and incomplete link counters, does a learned split-ratio policy beat a tuned reactive heuristic? We train a graph policy on 40 small synthetic graphs by optimising through a differentiable fluid model under corrupted observations, and apply it zero-shot to 24 held-out Topology Zoo graphs and replayed SNDlib traces, against a heuristic re-tuned per telemetry condition. Under the tested conditions the answer depends on telemetry quality: with clean or moderately impaired telemetry the heuristic matches or beats the learned policy; with delay, noise and drops together the learned policy is ahead (+0.033 dScore, 95% interval [+0.018, +0.049]; no loss on any of 24 graphs; ahead in all nine real-trace cells and for all five tuner seeds). In TCP emulation (27 scenarios) the simulator predicts a small lead and the emulation shows one of similar size, suggestive rather than established. A fallback guard halves simulated tail regret, not packet-level tail regret. Model-based fluid features, not message passing, carry the result; a single tuning run of the baseline can move a comparison as much as the effects reported. Impairments are modelled, not measured; poor telemetry does not by itself make a learned policy the better choice.

1 Introduction

Centralised traffic engineering (TE) sets split ratios over candidate paths from polled link counters, so every decision rests on telemetry that is delayed, noisy and sometimes missing. Learned TE policies are usually trained and tested on clean inputs, and the heuristics they are compared with are tuned once rather than for each telemetry condition. Prior work shows that neural routers degrade under realistic delay (Section 2), so we do not claim that observation. We ask a narrower, operational question:

> *For a given network and telemetry quality, does a learned policy that generalises zero-shot to unseen topologies outperform a heuristic re-tuned for the same conditions, what does each cost in tail risk and controller activity, and which components of the policy are responsible?*

What is and is not new. Not new: that delay degrades neural routing (LOGGIA [7], Roracle [6]), learned split-ratio TE (DOTE [1], FIGRET [2]) or falling back to a safe policy [10], [11]. This is an evaluation, not a new learning algorithm, and the combination is what is new: (i) one five-parameter telemetry channel (polling period, delay, noise, dropped counters, install latency) applied identically in a fluid simulator and to real OpenFlow counters in an emulated controller; (ii) a baseline protocol in which the heuristic is re-tuned for every impairment profile with five independent tuner seeds, so that rankings come with their stability under tuning; (iii) a zero-shot comparison on 24 held-out Topology Zoo graphs and replayed SNDlib traces, with a 27-scenario TCP emulation; (iv) the finding that the ranking depends on telemetry quality and on baseline tuning, while the learned policy's lead is small and not established at packet level; (v) an ablation showing that model-based fluid features, not message passing, carry the result; (vi) an ECMP-fallback guard that cuts tail regret in simulation but not at packet level; and (vii) an audit tool that applies the measurements to a user's network. We also report negative results (LP-teacher distillation, PPO fine-tuning). We do not claim that poor telemetry makes learned routing the better choice in general.

2 Related work

Learned TE. DOTE [1] and FIGRET [2] train networks on demand history to output split ratios over candidate paths by minimising maximum link utilisation; DOTE is the base paper for our method: we keep its idea of mapping demand history directly to split ratios and training by gradient through the objective, and we add a differentiable fluid model with a packet-loss term and training under impaired observations. They test demand noise or bursts, not stale or missing counters. FlexDATE [3] selects critical flows with RL and reroutes them by LP, assuming timely INT/SNMP telemetry; Teal [4] and CFR-RL [5] target scale or rerouting. *Staleness and delay.* Roracle [6] shows that LP-optimal routing computed on the current matrix degrades as the matrix ages (about 20% in MLU over five steps)

and that a learned lookahead resists this; its staleness is routing-computation delay and traffic drift, with no telemetry-impairment sweep and no tuned reactive baseline. LOGGIA [7] models communication and inference delay for neural link-weight routing in a packet-level simulator and reports that neural baselines stop beating static shortest-path routing under delay and that fully local deployment works best; it varies one impairment axis, compares with default shortest-path protocols and has no safety mechanism. EnvelopeTC [8] bounds edge-agent actions and disables reroutes when its policy envelope is older than a staleness limit, but staleness is a safety rule there, not a studied variable. RedTE [9] removes centralised control-loop latency by moving decisions to routers. *Safe fallback.* Training Wheels [10] falls back to a simple policy in unsafe states, and Safe Load Balancing [11] shields a DRL agent with a control barrier function against capacity violations; our guard applies the fallback idea. *Surveys and robust TE.* Deep-RL approaches to TE are surveyed in [12]. Robust TE under demand uncertainty predates learning: COYOTE [13] configures static traffic-splitting ratios that are optimised over operator-specified uncertainty bounds, for networks without online traffic measurement. Those works address demand uncertainty in the planned configuration, not the age and fidelity of the telemetry a reactive controller sees.

Production and classical TE. Centralised WAN TE with periodic measurement is deployed in SWAN [14] and B4 [15]; tuning link weights for shortest-path routing is the classical heuristic approach [16]; Applegate and Cohen [17] make intra-domain routing robust to uncertain demands; "Learning to Route" [18] was an early reinforcement-learning proposal and RouteNet [19] a graph-network model of network performance. We cite these as background and do not compare with them; to our knowledge none varies the age or fidelity of the controller's own measurements.

What is new. That delay hurts neural routing is not new (LOGGIA, Roracle). What none of the works in Table 1 combines is (a) centralised split-ratio TE, (b) a five-axis telemetry channel (polling period, delay, noise, dropped counters, install latency) applied identically in simulation and to real OpenFlow counters, (c) a heuristic baseline re-tuned for every impairment profile, with its tuning-seed variability reported, and (d) real topologies and traces with a packet-level check. The findings differ in kind: the ranking of learned and heuristic control depends on telemetry quality and on how the baseline is tuned, and the learned policy's lead is not established at packet level.

Table 1. Closest prior work compared with this study (entries checked against the cited texts; FlexDATE, CFR-RL and RedTE are not tabulated).

Work	Control setting	Staleness or impairment studied	Baselines	Safety mechanism
DOTE [1]	Split ratios over paths, minutes scale	Demand noise up to +-35%	LP-based TE schemes	None
FIGRET [2]	Split ratios over paths	Demand bursts; telemetry not varied	DOTE, desensitisation-based and LP-based TE	None
Teal [4]	Split ratios, WAN scale	Solver time only	LP solvers	None
Roracle [6]	Lookahead routing	Age of the traffic matrix (routing computation delay)	Optimal routing on the current matrix	None
LOGGIA [7]	Link weights, per-router or central, packet-level, sub-second	Communication and inference delay (one axis)	Shortest-path routing, neural baselines	None
EnvelopeTC [8]	Controller envelopes, edge agents	Staleness bound used as a safety rule	Static ECMP, centralised TE, threshold reroute, learned agents	Envelope, disable when stale
Safe LB [11]	DRL load balancing, packet-level simulator	Not varied	Non-RL load balancers	Control barrier function
This study	Split ratios, centralised, fluid model plus emulation	Polling period, delay, noise, missing counters, install latency	ECMP and a heuristic re-tuned per condition	Guard falling back to ECMP

Comparison with LOGGIA. We did not run LOGGIA, and a like-for-like comparison is not possible with the present setup: it optimises goodput by link-weight routing in a packet-level simulator at sub-second time scales, while we optimise maximum link utilisation by split ratios at the time scale of telemetry polls, in a fluid model. Our results are therefore a statement about split-ratio TE under impaired telemetry, not a ranking against LOGGIA. Adding our re-tuned heuristic and ECMP as baselines inside its public framework is future work.

3 Setting

Network and traffic. A topology with link capacities and k = 4 candidate paths per demand; the policy outputs split ratios each interval. Traffic is synthetic (gravity base, diurnal cycle, bursts, hotspots, shifts, flash crowds) or a replayed SNDlib trace [20] (Abilene, 5-min, 48,096 matrices; GEANT, 15-min, 11,460; Germany17 and Germany50, 5-min, 288 each; Brain, 1-min, 9,723, restricted to its 600 heaviest demands = 90% of sampled traffic). Each (network, load) pair uses one scale factor so that the median ECMP max-link-utilisation over the trace equals 0.6, 0.8 or 1.0. This is *real-trace evaluation under controlled load scaling*: temporal and spatial structure are real, absolute load is not. The scale factors are large and differ by network: to reach a median ECMP max-utilisation of 0.8 the Abilene and Brain traces are multiplied by about 9 and 23, GEANT by about 4, while the Germany17 and Germany50 traces are multiplied by about 0.02 (their native demands exceed the published module capacities by a factor of about 30-40, so capacity units in those files are not meaningful on their own). Some traces contain extreme intervals: the largest GEANT interval is about 12,000 times its median, so after scaling it has a max-utilisation near 10^4; Abilene's maximum is about 17 times its median. Such intervals dominate pooled tail statistics and are the likely source of the 'overloaded windows' of Section 6.4.

Telemetry impairment. The controller observes utilisation and demands through a channel with polling period P (intervals), delay d, multiplicative noise sigma and per-counter drop probability p; installing rules takes L intervals. Profiles: ideal (1,0,0,0), rich (1,1,0.03,0), SNMP-like (5,1,0.05,0.02), degraded (5,3,0.2,0.1). The same channel is applied to real OpenFlow counters in the emulated controller.

Where the profiles come from. They are stylised, not fitted: we had no operational telemetry logs. They are informed by published measurements: byte counters in the devices tested by Caria and Jukan [21] reach the MIB only every 0.5 s, 1 s or 10 s (a Cisco 2811 router by default), so polls shorter than that return stale values, and a case study of SNMP collection [22] found most link-load errors small but with a heavy tail, a few as large as the measurement. Our noise and drops are lighter-tailed than that, so Section 6.2c tests gross errors, lost poll rounds and poll jitter on top of the degraded profile. The profiles should be read as a sweep over axes, not as a calibration.

Metrics. We report separately: gain (reduction of mean max-link-utilisation vs ECMP), fluid packet loss, tail regret (p99 of per-interval max-util minus ECMP's; positive = worse than ECMP) and flow-update rate. The composite score (max-util ratio + 5 x loss, paired difference to ECMP, "dScore") is a convenience; conclusions do not depend on the weight (Section 6.1). **Unit of analysis:** for Zoo results, the graph (paired across methods; seeds and training seeds are averaged within a graph and reported as spread); for real traces, the (network, load) cell with independent trace windows inside it; windows within one day-long trace overlap and are not treated as independent.

4 Methods

Baselines and tuning budgets. *ECMP* and *static LP* (LP-optimal splits for the mean traffic matrix of the first 70% of a trace, no telemetry) have nothing to tune. *Tuned heuristic:* EWMA-smoothed bottleneck soft-min over candidate paths with hysteresis, damped updates and age-based shrinkage of stale readings; five parameters tuned per telemetry profile by random search, 80 trials per tuner seed, on four synthetic graphs disjoint from training and test graphs, repeated with five independent tuner seeds (100 to 104). The main comparisons average over tuner seeds, and Section 6.1 reports how often the verdict changes with the seed. The heuristic knows the telemetry profile, which favours it. *DOTE-style:* an MLP on the last three impaired demand estimates, trained per network on the first 70% of its trace (60 episodes per load at three loads, 25 epochs, Adam 1e-3, one seed, ideal- and degraded-telemetry variants) with our differentiable loss, and evaluated on the last 30%. *LP oracle:* perfect information, graphs with n <= 25 only; a reference bound, not a deployable method. The learned policy has a fixed schedule (12 iterations of 40 episodes at learning rate 1e-3, then 4 at 2e-4) and four training seeds; no baseline or model is selected on test graphs or windows.

Learned policy. A graph network on the link graph with path pooling (Section 6.3 shows the message passing is not what matters; a per-link MLP with the same inputs does as well or better). Inputs: impaired utilisation and demand history (H = 3), telemetry age, and model-based fluid features (ECMP-predicted utilisation, utilisation predicted for the current configuration, per-path what-if bottleneck). Training is direct optimisation through a differentiable fluid model: the policy sees the impaired observation, the loss uses the true next demand, `smoothmax(util)/ecmp_mu + 5 loss + 0.1 churn`. Training graphs are 40 synthetic BA/ER/geometric graphs with 10-20 nodes; no Zoo or SNDlib

graph is used. Variants: randomised-telemetry training ("ours") and ideal-only training; four seeds each. *Fine-tuning:* the pre-trained model is trained for 6 iterations (~3-5 minutes on CPU) on rollouts over the first 70% of a network's trace with randomised telemetry and evaluated on the last 30%.

Guard. L1: stale or missing telemetry, malformed action, deadline. L2: fluid-model check of the proposal against ECMP on the *estimated* state, rejecting it if predicted max-util exceeds ECMP's by more than a margin eps. L3: runtime monitor with backoff. The headline guard is L2 only with eps = -0.06, chosen on validation graphs. **Property claimed:** none formal. The guard is an empirically evaluated runtime filter; we report its fallback rate, false-rejection rate, the share of harmful proposals caught and the regret before and after (Section 6.5).

Rate limiter. A demand's installed split is kept unless the proposal moves it by more than a threshold (half the L1 distance); the number of updates per interval is the flow/group modifications a controller would send.

Audit tool. `stale-but-safe audit` takes a topology, optional traffic and telemetry parameters and reports best-case headroom (LP oracle where n <= 25), gain with a 95% bootstrap interval over trace windows and tail regret for each method under the user's telemetry, a polling sweep, and one of three outcomes: stay on ECMP, use a named method, or insufficient evidence (too few windows, interval straddling the 5% threshold, or gain concentrated in one window). "ECMP is enough" is declared only when the upper end of the 95% interval of the best method's gain under perfect telemetry is below a prespecified 5% max-utilisation tolerance, never from a non-significant difference alone; the 5% value is a practical choice that operators can change, not a derived constant. It flags telemetry values outside the evaluated range. Its output is an offline estimate, not a deployment recommendation.

5 Protocol

Topology Zoo [23]: 263 graphs split into train/validation/test; we report the 24-graph `test_subset`, 5 evaluation seeds per graph, horizon 100, load 0.8; paired bootstrap 95% CI, IQM [24] and Holm-corrected Wilcoxon over graphs. Real traces: four networks with headroom plus Brain, loads {0.6, 0.8, 1.0}, four profiles; two independent evaluation runs (4 windows, then 10 fresh windows per cell, 12 for Brain at load 0.8). Tuning and model selection never use test graphs or windows. Interval methodology: Zoo intervals are paired bootstrap over graphs on seed-averaged scores, so they capture graph variability but not training-seed variability (reported separately as seed SD, 0.001-0.002; the GNN-vs-MLP and fluid-feature comparisons use four matched training seeds each, averaged per graph before resampling). Real-trace intervals are paired bootstrap over evaluation windows for one trained model and one trace, so they do not generalise to other days or networks. The 5% ECMP tolerance is applied identically everywhere: "ECMP is enough" requires the interval's upper end to be below 5%; a lower end below 5% gives "insufficient evidence". Evaluation uses 5 seeds per Zoo graph and 12 to 16 random windows per real-trace cell, fewer than the 20 seeds we planned. We train four policy seeds, not ten, and some ablations use one seed. Several baselines and analyses were added after the first results (the re-tuned heuristic, the delay grid and the window-level intervals), and a list of deviations from the original protocol is in `docs/eval_protocol.md` in the repository.

6 Results

6.1 Held-out Topology Zoo graphs (24 graphs)

The heuristic is tuned separately for each telemetry profile. For every profile we run five independent tuner seeds with 80 trials each on the same four synthetic tuning graphs, and evaluate each resulting parameter set on the 24 held-out graphs with the same evaluation seeds as the learned policy. The learned policy is the mean of four training seeds. We report the heuristic averaged over the five tuner seeds, because the best of five is an optimistic estimate and seed variance is a known hazard in learned control [25]; the spread over tuner seeds is given as a range. A "tie" on a graph means the two dScores differ by less than 0.002, about 3% of the learned policy's mean dScore. All intervals are percentile bootstrap intervals over graphs (4000 resamples, fixed seed), so they measure graph-to-graph variability and not tuner-seed variability.

Table 2. Learned policy versus the re-tuned heuristic on 24 held-out Zoo graphs. dScore is the score change relative to ECMP (lower is better). Differences are heuristic minus learned, so positive values favour the learned policy.

W/T/L counts graphs where the learned policy wins, ties and loses. Tail columns use the per-graph p99 regret.

telemetry	learned dScore	heuristic dScore (range over tuner seeds)	difference [95% CI]	W/T/L	p99 regret learned / heuristic	tail difference [95% CI]	graphs with lower learned p99
ideal	-0.065	-0.076 (-0.085 to -0.065)	-0.011 [-0.026, +0.002]	4/11/9	0.018 / 0.036	+0.018 [+0.000, +0.039]	9 of 24
rich	-0.063	-0.076 (-0.081 to -0.073)	-0.014 [-0.027, -0.004]	3/10/11	0.018 / 0.036	+0.018 [+0.004, +0.035]	10 of 24
SNMP	-0.062	-0.060 (-0.072 to -0.053)	+0.002 [-0.007, +0.012]	7/10/7	0.017 / 0.077	+0.060 [+0.027, +0.101]	13 of 24
degraded	-0.061	-0.029 (-0.045 to -0.004)	+0.033 [+0.018, +0.049]	15/9/0	0.019 / 0.092	+0.073 [+0.038, +0.113]	13 of 24

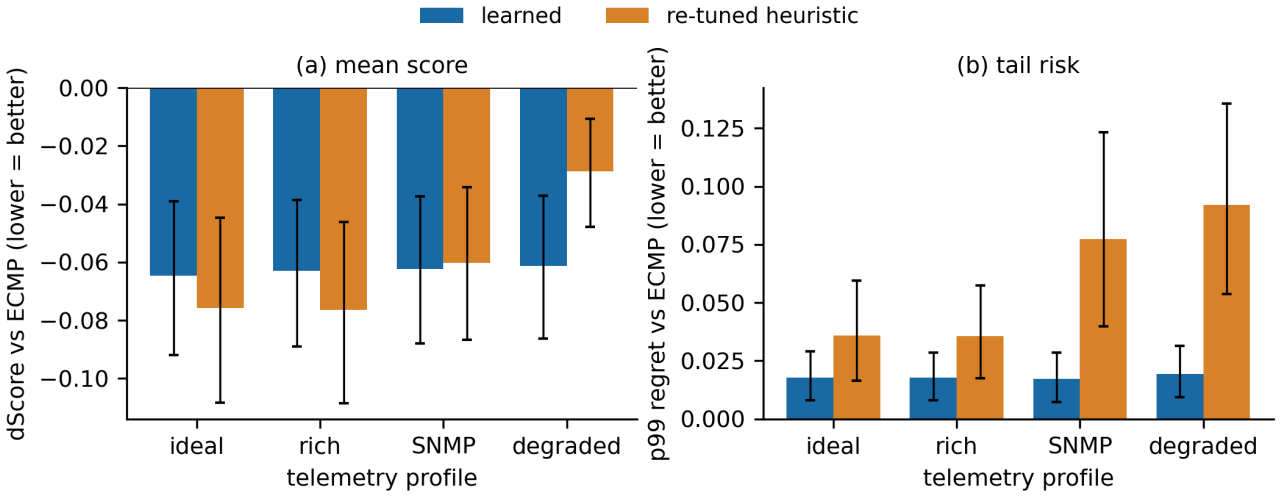


Figure 1. Held-out Zoo graphs (24). (a) Mean dScore and (b) mean per-graph p99 regret by telemetry profile for the learned policy (mean of 4 training seeds) and the re-tuned heuristic (mean of 5 tuner seeds). Error bars are 95% bootstrap intervals over graphs.

Table 2 and Fig. 1 give three results.

Average performance depends on telemetry quality. With ideal or rich telemetry the re-tuned heuristic is better on average, and the difference excludes zero for rich telemetry. With SNMP-like telemetry the two are tied (7 wins, 10 ties, 7 losses). Under degraded telemetry the learned policy is ahead on average and wins 15 graphs, ties 9 and loses none. Seed-to-seed variation of the learned policy is small (standard deviation 0.001 to 0.002).

Tail risk is lower for the learned policy on average, but not on every graph. Its mean p99 regret is lower in every profile, and the interval of the difference excludes zero for rich, SNMP and degraded telemetry (for ideal telemetry the lower end is 0.000). However, the learned policy has the lower p99 on only 9 to 13 of the 24 graphs. The average gap comes from a minority of graphs on which the heuristic has a large tail. We therefore describe the advantage as lower on average, not as a uniform improvement.

Tuning variation is large. Under SNMP telemetry the heuristic's dScore ranges from -0.072 to -0.053 across tuner seeds, which is as large as the differences in Table 2. A comparison based on one tuning run can therefore change sign. Repeating the comparison for every tuner seed, and for each of the four training seeds separately (20 pairs), the verdict never reverses from one side to the other at any profile: under degraded telemetry the learned policy is ahead for all 5 tuner seeds (20 of 20 pairs); under SNMP telemetry all 5 seeds give a tie, although the sign of the mean difference changes with the seed (-0.010 to +0.009); under ideal and rich telemetry the heuristic is ahead for 3 and 4 of the 5 seeds and tied for the rest (`table_ranking_stability.md`). The ranking is therefore stable at the two ends of the telemetry range, and tuner-seed variability decides only whether the middle is a tie.

The ranking does not depend on the loss weight in the score. At weight 0 (pure maximum utilisation) the heuristic still wins at ideal telemetry (dScore -0.061 against -0.043) and the learned policy wins at degraded telemetry (-0.040

against -0.030). At weight 20 the learned policy wins on 21 of 24 graphs under degraded telemetry, and the heuristic stays ahead under ideal, rich and SNMP telemetry. The heuristic is tuned with weight 5 in all cases.

We also tested four stress scenarios on 12 graphs under SNMP and degraded telemetry: link failures, demand shifts, flash crowds and hotspots (Table 3).

Table 3. Stress scenarios on 12 Zoo graphs. dScore and p99 regret as in Table 2; difference is heuristic minus learned with a 95% bootstrap interval over graphs.

scenario	telemetry	learned dScore	heuristic dScore	difference [95% CI]	learned W/T/L	p99 regret learned / heuristic
link failure	SNMP	-0.066	-0.070	-0.004 [-0.018, +0.009]	3/6/3	0.018 / 0.065
link failure	degraded	-0.064	-0.044	+0.020 [+0.007, +0.036]	6/6/0	0.019 / 0.081
demand shift	SNMP	-0.059	-0.047	+0.011 [+0.001, +0.027]	4/7/1	0.044 / 0.101
demand shift	degraded	-0.057	-0.021	+0.035 [+0.015, +0.058]	7/5/0	0.045 / 0.143
flash crowd	SNMP	-0.073	-0.067	+0.005 [-0.009, +0.023]	5/4/3	0.022 / 0.078
flash crowd	degraded	-0.069	-0.036	+0.032 [+0.012, +0.059]	7/5/0	0.028 / 0.105
hotspot	SNMP	-0.126	-0.090	+0.036 [-0.012, +0.105]	4/2/6	0.042 / 0.116
hotspot	degraded	-0.123	-0.022	+0.101 [+0.029, +0.204]	9/3/0	0.048 / 0.167

Under degraded telemetry the learned policy is ahead in all four scenarios, with intervals that exclude zero. Under SNMP telemetry the intervals include zero except for demand shifts. The learned policy has lower p99 regret in all eight cells, but each cell has only 12 graphs and the caveat about a minority of graphs applies here too. In this table the heuristic is the parameter set with the best tuning score for each profile.

6.2 Real traces

We replay SNDlib traces on Abilene, GEANT and Germany17 at three target loads (0.6, 0.8 and 1.0) and four telemetry profiles. Each cell uses 12 random 100-interval windows. For each cell we bootstrap the paired difference in gain (learned minus heuristic) over windows. The heuristic is the parameter set with the best tuning score for each profile (Section 6.1). Windows with zero traffic are skipped.

Table 4. Real-trace comparison, 36 cells (3 networks, 3 loads, 4 profiles, 12 windows each). Counts show cells where the paired 95% interval favours the learned policy, includes zero (tie) or favours the heuristic. The last column gives the range of the mean difference in points of gain (positive favours the learned policy).

telemetry	learned ahead	tie	heuristic ahead	range of mean difference
ideal	0	0	9	-7.7 to -2.4
rich	0	3	6	-4.7 to +0.9
SNMP	3	4	2	-3.5 to +4.6
degraded	9	0	0	+7.8 to +13.0
all	12	7	17	

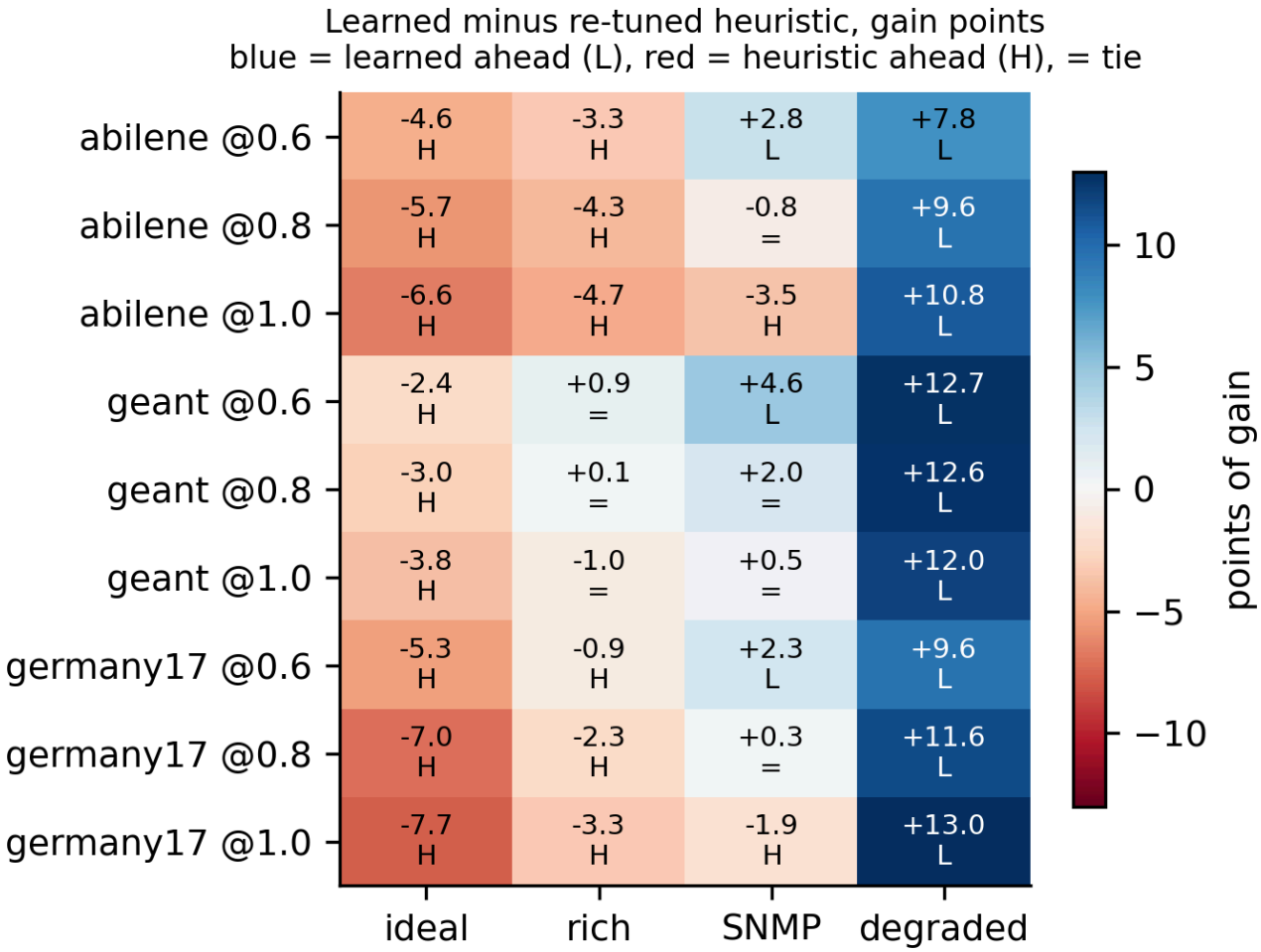


Figure 2. Real traces: learned minus heuristic gain in points, per network, load and telemetry profile. Letters give the verdict of the paired 95% bootstrap interval over windows (L learned ahead, H heuristic ahead, = tie).

The oracle gain (for graphs with at most 25 nodes) is 47% to 58% on Abilene and 37% on Germany17, and the zero-shot learned policy captures roughly half of it. The real-trace results agree with the Zoo results. Under ideal telemetry the heuristic is ahead in every cell. Under rich telemetry it is ahead or tied. Under SNMP-like telemetry the result is mixed. Under degraded telemetry the learned policy is ahead in all nine cells, by 7.8 to 13.0 points. On Germany17 the heuristic's gain under degraded telemetry is only 1.5% to 2.8%, while the learned policy gains 12% to 14%. Germany50, a 50-node network larger than any training graph, and the Brain network were evaluated only with a single-run heuristic; see Appendix A. In the Brain network the heaviest 600 demands carry 90% of the traffic, and gains are about 0.3% for every method in most windows, with one rare window in which the learned policy gained 20.9%.

6.2a Which impairment drives the ranking?

Delay and the heuristic's parameters. We first vary only the telemetry delay d, with everything else ideal, and run the heuristic in three ways: with parameters frozen at the values tuned for clean telemetry, frozen at the values tuned for degraded telemetry, and re-tuned at each delay (one 40-trial run on four synthetic graphs). The learned policy is the mean of four training seeds. Mean gain over ECMP at load 0.8 for d = 0, 3, 5 and 10 on Abilene: frozen clean-tuned parameters 34.0, 6.4, 0.0, 0.0%, re-tuned 41.9, 34.7, 23.2, 21.0%, learned 34.5, 34.2, 34.5, 34.7% (GEANT: frozen 37.6, 1.9, 0.0, 0.0%, re-tuned 45.6, 43.6, 23.2, 24.9%, learned 42.6, 42.5, 42.5, 41.8%). A heuristic with frozen parameters collapses to about zero gain at delays of five intervals or more. This mostly reflects mis-tuning: when re-tuned at each delay, the heuristic keeps most of its gain. It matches the learned policy at small delay (at d = 3: Abilene 34.7 against 34.2, GEANT 43.6 against 42.5) and is behind at d = 5 and d = 10. The learned policy's gain

does not depend on d. Because a single tuning run is noisy, the comparisons in the rest of this paper use re-tuned heuristics, and the grid below uses repeated tuning.

Delay and polling period together. We re-tune the heuristic five times in each (d, P) cell, with independent tuner seeds and 80 trials each, where P is the polling period. The reported heuristic is the seed with the best tuning score, so selection never uses evaluation data. Table 5 shows the learned policy's gain minus the heuristic's, in points.

Table 5. Learned minus re-tuned heuristic gain (points) over the delay d and polling period P grid. Positive favours the learned policy. Zoo values average 10 graphs; Abilene and GEANT use the same evaluation windows as the delay comparison above.

case	d=0 P=1	d=0 P=5	d=0 P=20	d=3 P=1	d=3 P=5	d=3 P=20	d=10 P=1	d=10 P=5	d=10 P=20
Zoo	-0.4	-0.1	+0.9	+1.2	+1.0	+1.1	+1.5	+1.9	+0.9
Abilene	-7.2	-2.9	+5.4	-2.7	-1.8	+8.5	+3.5	+6.3	+6.2
GEANT	-1.7	+1.8	+12.5	+0.7	+3.8	+15.5	+5.0	+11.3	+15.0

The heuristic is ahead in 5 of 18 real-trace cells (Abilene with d at most 3 and P at most 5; GEANT at d = 0, P = 1), within one point in one more, and behind in the rest by up to 15.5 points. Even against the best of the five tuner seeds, an oracle on evaluation data that favours the heuristic, the learned policy stays 2.9 to 9.5 points ahead in all ten cells with P = 20 or d = 10. On Zoo graphs the two are within 0.4 points at d = 0 with P at most 5, and the learned policy is ahead by 0.9 to 1.9 points elsewhere. The learned policy's gain varies by less than two points across the grid, while the heuristic's gain falls with both delay and polling period (Abilene: 41.7% at d = 0, P = 1 down to 27.8% at d = 10, P = 20). The spread over tuner seeds inside one cell is up to about 12 points (GEANT at d = 10, P = 1: 24.4 to 36.8), so margins smaller than that should not be over-read. The pattern is a gradient. No delay value flips the ranking independently of the polling period and the network.

Window-level uncertainty. The selected heuristic is evaluated on 16 fresh windows per cell (14 on GEANT after skipping zero-traffic windows), paired with the mean of four learned seeds, with bootstrap intervals over windows (Fig. 3).

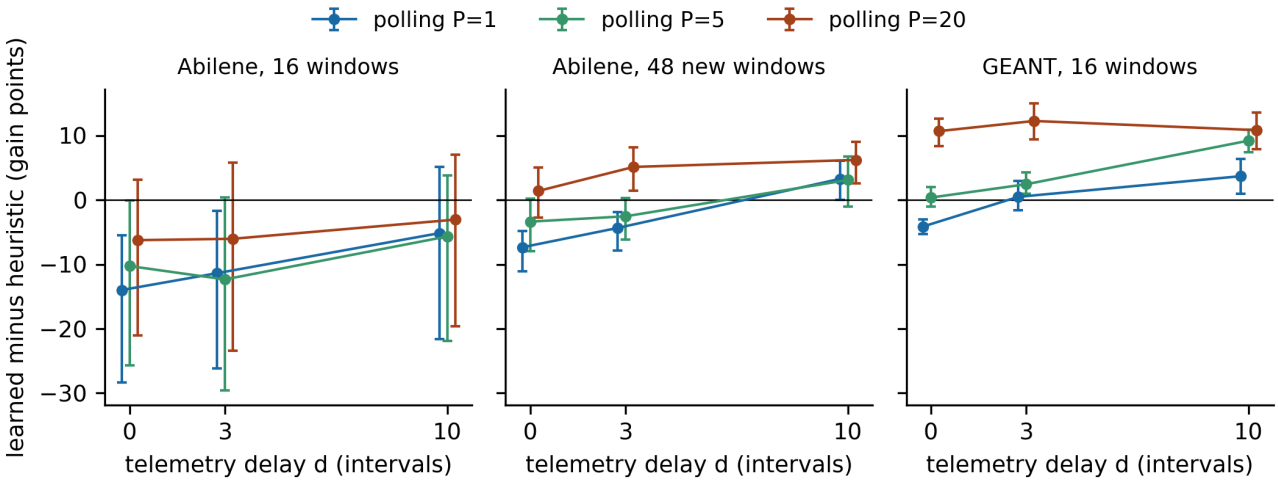


Figure 3. Delay and polling grid with the re-tuned heuristic: learned minus heuristic gain in points, with 95% bootstrap intervals over windows. Left: Abilene, the 16-window sample. Middle: Abilene, the 48 additional windows. Right: GEANT, 16 windows (14 after skipping zero-traffic windows). The two Abilene panels use different window samples, so they are not repeated measurements of one estimate.

On GEANT the pattern is clear. The heuristic is ahead only at d = 0, P = 1 (-4.1 [-5.2, -3.0]), two cells are ties, and the learned policy is ahead in the other six, by 2.5 to 12.3 points. Its advantage grows with delay and polling period. On Abilene this 16-window sample did not resolve the result. The heuristic's point estimate is ahead in all nine cells, by 3.0 to 14.0 points, the interval excludes zero in three cells (heuristic ahead) and includes zero in six, and the interval half-widths are 10 to 15 points. We therefore ran 48 further windows. The tuner saw only synthetic graphs and never a real trace or evaluation window.

Why the Abilene result depends on the sample. With identical policies and telemetry (clean, d = 3, P = 5) the learned policy is level with the heuristic on one window sample (12 windows) and 12.3 points behind on another (16 windows), and its own gain moves from 36% to 29%; the heuristic's parameters matter too (under degraded telemetry the grid-selected set reaches 31.2% against 17.0% for the per-profile set). We found no implementation inconsistency and cannot attribute the rest. Because the 16-window result could not be resolved, we evaluated the same selected heuristic on 48 new windows per Abilene cell (`experiments/final/abilene_more.json`).

With 48 windows the Abilene pattern resembles GEANT's. The heuristic is ahead with fresh and frequent telemetry (d = 0 and d = 3 with P = 1), the learned policy is ahead in three cells with stale or infrequent telemetry (d = 3 with P = 20, d = 10 with P = 1, and d = 10 with P = 20, by 3.3 to 6.2 points), and four cells are ties. The learned policy's gain is stable across the grid (34.6% to 36.5%), while the heuristic's gain falls from 43.1% to 30.0%. The earlier 16-window sample showed the heuristic ahead everywhere, so Abilene results depend on the window sample, and we treat 16 windows as too few for this network. Even with 48 windows several cells remain ties, and the ranking in a single cell should not be over-read.

6.2b Sensitivity to the load-scaling rule

We scaled each real trace so that the median, mean or 95th percentile of the ECMP max-utilisation equals 0.8 and ran Abilene, GEANT and Germany17 under ideal and degraded telemetry on 6 windows each (18 cells; no intervals, a win means a gain difference above one point). Under ideal telemetry the heuristic is ahead in all nine cells (by 3.6 to 9.9 points); under degraded telemetry the learned policy is ahead in all nine (by 1.2 to 13.7; smallest margin Abilene with the 95th-percentile rule, 20.3% against 21.5%). The conclusion does not depend on the scaling rule, although the absolute numbers do.

6.2c Impairments closer to measured collection

The stylised profiles lack heavy-tailed errors, lost poll rounds and timing jitter. On 10 Zoo graphs (n <= 25) we added each to the degraded profile (2% or 5% of readings replaced by the reading scaled by U(0, 2); 10% of poll rounds lost; poll interval jittered by one interval with probability 0.3) and all three together, without retraining the learned models. The learned policy's gain stays at 4.1% to 4.2% and it stays ahead of the heuristic by 1.9 to 2.8 points (every interval excludes zero), with p99 regret 0.059 to 0.079 against 0.261 to 0.477. Re-tuning the heuristic for the two harsher variants (two tuner seeds) does not close the gap (3.5 and 3.2 points). This is a Zoo-only check at degraded telemetry, and the heuristic's absolute gain is small there (0.7% to 2.3%), so it supports robustness to these impairment types, not a calibration to any network (`experiments/final/table_telemetry_realism.md`).

6.3 What carries the result: baselines and ablations

Table 6 compares the zero-shot policy with models that use a network's own history, on held-out windows (the last 30% of the trace).

Table 6. Held-out gain (%) on Abilene at load 0.8. The heuristic column is the single-run heuristic of Appendix A, not the re-tuned one.

Abilene @0.8	single-run heuristic	static LP	DOTE-style (degraded-trained)	GNN zero-shot	GNN fine-tuned
ideal	33.6	34.5	49.1	23.7	46.2
SNMP	27.7	34.5	49.1	22.4	41.3
degraded	23.5	34.5	48.9	22.5	39.3

On GEANT the DOTE-style model reaches about 18% in every profile, the zero-shot policy 15-18%, the fine-tuned policy 16-19% and static LP -38.6% (traffic is not stationary). Hence: (a) with per-network history a DOTE-style model beats the zero-shot GNN, and fine-tuning recovers most of the difference on Abilene but little on GEANT; (b) robustness to telemetry is a property of any policy that does not rely on fresh counters, not specific to our GNN; (c) what the zero-shot policy offers is no need for per-network training data. Ablations (24 Zoo graphs; one seed for the history and age ablations, and four training seeds each for the full model, the per-link MLP and the no-fluid-feature model): removing history costs +0.003 at ideal telemetry only; removing the age input is ~0. **Matched-seed GNN vs MLP:** the per-link MLP (no message passing) has mean dScore -0.068 (ideal) / -0.063 (degraded) against -0.065 /

-0.061 for the full graph model; seed-averaged paired difference -0.0031 [-0.0054, -0.0011] and -0.0020 [-0.0040, -0.0002], the MLP better on 14 of 24 graphs in both profiles. Its p99 regret is slightly worse under degraded telemetry (0.022 vs 0.019). So message passing does not help; the effect sizes are small and we claim only "no benefit", not that MLPs are better. **Fluid-model features:** removing them costs +0.0082 [+0.0048, +0.0121] (ideal) and +0.0144 [+0.0085, +0.0206] (degraded) and worsens p99 regret (0.018 to 0.029 degraded); the full model is better on 21-22 of 24 graphs. The model-based features, not the graph architecture, carry the result. Training with randomised telemetry versus ideal-only: the mean differences are within seed noise (~0.002), but the randomised model has a lower p99 regret under degraded telemetry (0.019 vs 0.027 over four seeds).

6.4 Where it fails: overload

Real-trace windows in which ECMP's own mean max-utilisation exceeds 2 are traffic surges far above the requested load, and they lie outside the training range (loads 0.5 to 1.1). We compared the methods on 30 Abilene windows at load 0.8 (Table 7). Of these, 27 are typical and 3 are overloaded.

Table 7. Typical and overloaded Abilene windows at load 0.8 (30 windows). Gain over ECMP in %: median and worst window.

telemetry	window type	windows	heuristic gain (median, worst)	learned gain (median over seeds, worst)	learned + guard (median, worst)	learned worse than ECMP in
ideal	typical	27	41.1, 8.6	36.5, 2.1	28.6, 0.0	0% of windows
ideal	overloaded	3	49.0, 48.3	9.1, 4.6	19.9, 18.3	33% of windows
degraded	typical	27	28.6, -21.5	35.3, 1.4	29.0, 0.1	0% of windows
degraded	overloaded	3	42.5, 40.9	20.8, 6.1	26.8, 25.6	17% of windows

In typical windows the heuristic is better under ideal telemetry, and the learned policy is better and never worse than ECMP under degraded telemetry, where the heuristic's worst window loses 21.5% against ECMP. In the three overloaded windows the heuristic gains 41% to 49% and the learned policy 9% to 21%, and the learned policy is worse than ECMP in one of the three. The guard lifts the learned policy to 20% to 27% in these windows. Tail statistics pooled over windows are therefore dominated by a few surges.

6.5 Guard

Definitions (per interval, using the true demand, simulator only): *beneficial* proposal = true max-util below 0.99 x ECMP's, *harmful* = above 1.01 x; false rejection = beneficial proposals among fallbacks; harm caught = harmful proposals replaced. Table 8 reports the headline guard (layer 2 only, eps = -0.06) on 10 Zoo graphs with at most 25 nodes.

Table 8. Guard behaviour on 10 Zoo graphs. Fallback is the share of intervals replaced by ECMP; false rejection is the share of fallbacks where the proposal was beneficial; harm caught is the share of harmful proposals replaced; gain kept compares the guarded with the unguarded gain.

scenario	fallback	false rejection	harm caught	gain kept	p99 regret raw -> guarded
ideal	71%	12%	43%	83%	+0.050 -> +0.019
degraded	72%	14%	22%	76%	+0.065 -> +0.057
link failure, degraded	80%	13%	90%	91%	+0.148 -> +0.021
demand shift, degraded	73%	18%	53%	74%	+0.084 -> +0.058

On real Abilene and GEANT traces (load 0.8, three typical windows each) the guard fires in only 0-2% of intervals because the learned policy's proposals are essentially never harmful there (0.0% of accepted intervals harmful; raw p99 regret already <= 0); its value on real traces appears in overloaded windows (Section 6.4). On the 24 test graphs the per-graph p99 regret falls from 0.019 to 0.008 at degraded telemetry (average of per-graph p99), while the pooled p99 above falls only from +0.065 to +0.057: the guard cuts the typical tail but misses harmful proposals concentrated in a few graphs. The guard is most effective under link failures. It falls back often (71% to 80%) yet keeps 74% to 91% of the gain because fallback occurs mostly in low-gain intervals. On real traces the validation-chosen guard barely affects the learned policy (0.0 to 0.8 points of gain).

Table 9 applies the same guard to the learned policy and to the re-tuned heuristic on all 24 Zoo graphs.

Table 9. Guard applied to the learned policy and to the re-tuned heuristic (24 Zoo graphs, layer 2 only, eps = -0.06). Each cell gives mean dScore and mean per-graph p99 regret.

telemetry	learned	learned + guard	re-tuned heuristic	heuristic + guard
ideal	-0.064, 0.017	-0.054, 0.008	-0.085, 0.026	-0.064, 0.008
rich	-0.062, 0.018	-0.051, 0.009	-0.073, 0.041	-0.058, 0.019
SNMP	-0.062, 0.017	-0.051, 0.010	-0.072, 0.056	-0.054, 0.031
degraded	-0.060, 0.019	-0.049, 0.008	-0.040, 0.093	-0.042, 0.030

The guard roughly halves the p99 regret of both policies (learned 0.017-0.019 to 0.008-0.010; heuristic 0.026-0.093 to 0.008-0.031) and its fallback rate is 78-84%. Its cost in average dScore is about +0.011 for the learned policy in every profile and +0.015 to +0.021 for the heuristic at ideal, rich and SNMP telemetry, but essentially nothing for the heuristic under degraded telemetry (-0.040 to -0.042), where it lowers p99 regret from 0.093 to 0.030. With the guard, the learned policy and the re-tuned heuristic have similar p99 regret at ideal and rich telemetry (0.008 vs 0.008, 0.009 vs 0.019) and the learned policy is lower elsewhere. The guard is therefore a tail-risk trade against mean performance for either policy, not a property that favours the learned one. These are simulator results: in the TCP emulation the guard cost the same mean utilisation but did not reduce the tail (Section 6.6).

6.6 Operational cost and emulation

Without a rate limiter the learned policy updates 91% of demands every interval in simulation (Zoo, ideal telemetry); a 5% threshold cuts this to 8% and a 10% threshold to 2.4% with unchanged gain (3.1%). On the real traces (degraded telemetry) the learned policy updates 98.5% (Abilene) and 100% (GEANT) of demands per interval without a limiter; thresholds of 5%, 10% and 20% cut this to 5.1/1.7/0.9% (Abilene) and 6.3/1.9/0.8% (GEANT) while the gain stays or rises (Abilene 26.4 -> 27.6/32.4/36.8%, GEANT 37.9 -> 38.2/38.5/37.7%; thresholds of 0.5 lose gain).

Emulation: Mininet [26] with Open vSwitch, OpenFlow [27] and a Ryu controller reading real OpenFlow counters, 24 flows, four scenarios (three traffic seeds on a 10-node graph, one 12-node random graph), six policy/telemetry runs each plus a run with a 20% update threshold (28 runs). Simulator and emulator agree: across the 24 runs at the default 5% threshold the correlation between emulated and simulated mean max-utilisation is 0.99, the mean absolute difference 0.017 and the bias +0.010; per-run time-series correlation is 0.83-0.98; the policy ranking agrees (Kendall tau 0.47-0.73 per scenario). Table 10 gives means over the four scenarios.

Table 10. Emulation results, means over four scenarios: emulated max-utilisation, flow-mods per minute and controller loop time.

policy [telemetry]	max-util	flow-mods/min	loop ms
ECMP	0.866	0	9
re-tuned heuristic [SNMP]	0.772	140	14
re-tuned heuristic [degraded]	0.806	139	14
learned [SNMP]	0.763	270	12
learned [degraded]	0.772	332	14
learned [degraded], threshold 0.2	0.790	76	12
learned + guard (L2, eps -0.06) [degraded]	0.794	258	12

The re-tuned heuristic runs agree with the simulator as well (8 runs, correlation 1.00, mean absolute difference 0.018). Under SNMP telemetry the re-tuned heuristic (0.772) is close to the learned policy (0.763), but it reprograms about 140 flows per minute against 270 to 332 for the learned policy, so the learned policy costs about twice the flow updates. Under degraded telemetry the learned policy is ahead (0.772 against 0.806, loss 0.0080 against 0.0095) in all four scenarios, by only 0.001 in one of them. Under SNMP the two are level on average; the learned policy is lower in three scenarios and higher in one. A 20% update threshold cuts the learned policy's updates 4.4-fold (332 to 76 per minute) at a cost of 0.018 in max-utilisation, leaving it ahead of the re-tuned heuristic under degraded telemetry (0.790 against 0.806). In the emulator the guard raises max-utilisation and packet loss (0.794 against 0.772 max-utilisation, and loss 0.0143 against 0.0080 for the unguarded policy). Loop times were measured with the host otherwise idle. Capacities are scaled down and traffic is synthetic, so absolute rates do not transfer.

Packet-level check with TCP. The runs above use paced UDP. We repeated a reduced experiment with long-lived TCP connections (16 per demand, application-limited at the scheduled rate, so the kernel's congestion control decides what is delivered), 60 demands, 90 one-second intervals, degraded telemetry, first on nine scenarios (four 16-node Barabasi-Albert graphs, three 14-node geometric graphs, two Abilene scenarios) with three runs per policy. Means over the nine scenarios: goodput shortfall 0.0247 for ECMP, 0.0126 for the re-tuned heuristic and 0.0164 for the learned policy; max-utilisation 0.934, 0.902 and 0.896. Both reactive controllers beat ECMP (learned 0.039 lower, bootstrap interval [0.020, 0.059], 9 of 9 scenarios, Wilcoxon p = 0.004; heuristic 0.032 lower, 8 of 9, p = 0.02), but the learned policy and the heuristic are not distinguishable (max-utilisation 0.0065 lower, interval [-0.019, +0.003], 6 of 9, p = 0.50; shortfall 0.0038 higher, [-0.005, +0.017]). Run-to-run variation is not small: the within-scenario standard deviation of max-utilisation is 0.0040 for ECMP, 0.0059 for the heuristic and 0.0111 for the learned policy, and the same comparison from the first run alone is +0.0020 instead of -0.0065, so single-run comparisons should not be trusted. The learned policy reprograms about 448 flows per minute against 94 for the heuristic; with a 20% update threshold it needs 56 and is level with the heuristic (0.904 against 0.902). Emulated and simulated max-utilisation correlate at 0.97 across the 36 first-run measurements.

Why nine scenarios could not settle it. A simulator replay of the same nine scenarios (same demand schedule, capped counters) predicts a learned-minus-tuned max-utilisation of only -0.0063 [-0.0146, +0.0010], 7 of 9 lower, p = 0.13, and the emulation measured -0.0065 [-0.0194, +0.0033]: the emulation agrees with the simulator, and both intervals are wider than the effect. About 28 scenarios would be needed to detect it. We therefore declared 18 further scenarios in advance (design, primary outcome and simulator prediction committed before any run; `docs/tcp_batch2_design.md`): gravity traffic at load 0.6, 0.8 and 1.0, hotspots, demand shift and flash crowd on Barabasi-Albert and geometric graphs, and two Abilene scenarios, three runs per policy each. The simulator predicted -0.0087 [-0.0156, -0.0030]. The emulation measured -0.0056 [-0.0116, -0.0006] (lower in 12 of 18, Wilcoxon p = 0.18; shortfall -0.0046 [-0.0109, -0.0007]) and, over all 27 scenarios, -0.0059 [-0.0119, -0.0009] (18 of 27, p = 0.11; shortfall -0.0018 [-0.0071, +0.0042]). Per-scenario agreement with the simulator is moderate (Pearson r = 0.78 on the 18, 0.47 on all 27). The advantage sits where the simulator expects it: gravity traffic at load 0.8 (emulated -0.016, simulated -0.016) and demand shifts (-0.014, -0.019), whereas in the hotspot and load-1.0 scenarios the learned policy gains nothing over ECMP, and on the two Abilene scenarios it is 0.004 above the heuristic. We read this as a small advantage of the size the simulator predicts; the bootstrap interval excludes zero but the rank test does not, so it is suggestive, not established.

The guard at packet level. The guard (eps = 0.05, all layers) raises max-utilisation over the unguarded learned policy in every scenario (+0.019, interval [0.009, 0.032], p = 0.004) without a detectable change in shortfall (-0.002, p = 0.13); the pre-action variant (eps = -0.06, one run per scenario) raises it by 0.022 ([0.008, 0.039], 1 of 9 lower). It replaces the policy with ECMP in 5% to 98% of intervals (35% on average for eps = 0.05, mostly through the runtime monitor, which latched for 55 of 89 intervals in one scenario where the learned policy was clearly ahead of ECMP, 0.886 against 0.957), and the eps = -0.06 variant sat on ECMP for 98% of intervals in four scenarios. It does not flap (2 to 10 switches between the two controllers per 89 intervals, except 30 in one scenario); the extra max-utilisation equals the learned gain given up in the rejected intervals (+0.0157 against +0.0150). Replaying the same nine scenarios in the simulator with the emulator's capped counters gives the same mean cost: guarded minus unguarded learned max-utilisation is +0.017 in simulation and +0.016 in emulation, higher in 7 and 8 of 9 scenarios. What differs is the tail: pooled p99 regret against ECMP falls from 0.083 to 0.033 in the replay but only from 0.095 to 0.091 in the emulator, and the share of intervals worse than ECMP by more than 0.02 is 9.2% unguarded and 10.7% guarded there (2.0% and 1.3% in simulation). The emulated tail is mostly events the fluid model does not predict: the guard's estimate of the proposal's improvement over ECMP correlates at -0.12 to +0.02 with the improvement measured in instrumented runs. That estimate is weakly informative in the simulator too (0.09), so we did not isolate where the simulated tail gain comes from. A 20% update threshold does not help (one run per scenario; guard 0.921, pre-action variant 0.920, unguarded 0.904; the guard is higher in 9 of 9 scenarios, +0.016 [0.009, 0.024], p = 0.004), and the guard still reprograms 238 flows per minute against 56. In the TCP emulation the guard therefore buys nothing.

The controller also has a shadow mode, in which the policy runs on real counters but nothing is installed. On three scenarios it issued no flow updates and max-utilisation stayed at the ECMP level; its fluid-model prediction was

below ECMP's in one scenario (0.717 against 0.825) and above it in another (1.396 against 1.183), so the prediction is a diagnostic, not a promise.

Decision latency (single CPU thread, p50 and p99 over 100 intervals): the learned policy takes 2.1 / 4.4 ms on a 12-node graph (132 demands), 11.8 / 19.7 ms at 25 nodes (600 demands), 84 / 114 ms at 50 nodes (2,450 demands) and 311 / 412 ms at 80 nodes (6,320 demands), against 0.3 to 23 ms for the tuned heuristic. Cost grows roughly with links x demands, which is fine for minute-scale control intervals but not for sub-second loops on networks above about 50 nodes with all-pairs demands.

6.7 Audit tool

We ran the audit tool on four real networks under degraded telemetry (6 trace windows, re-tuned heuristic). It recommends the tuned heuristic for Abilene (30.6%, 95% interval [27.2, 34.1]; headroom 56.5%), which it cannot distinguish from the learned policy; the learned policy for GEANT (37.5% [23.1, 51.0], warning that the heuristic is worse than ECMP in 7% of intervals) and Germany50 (33.9% [30.7, 36.8]); and ECMP for Brain, flagged provisional because six windows cannot rule out the rare surge window in which the learned policy gained 21%. With the single-run heuristic the Abilene and GEANT recommendations were the opposite, so close calls depend on the heuristic's tuning and call for more windows. It remains an offline estimate; its verdicts have not been validated against any operator decision.

7 Negative results

LP-teacher distillation (DAgger) produced a weak student (6.4% gain vs 15.0% for the tuned heuristic on training graphs); PPO fine-tuning did not change it; direct optimisation through the fluid model fixed this. Randomised-telemetry training did not improve mean performance over ideal-only training. A per-link MLP did as well as the GNN, with four matched seeds each (Section 6.3). Static LP routing is excellent on a stationary network (Abilene) and disastrous on a non-stationary one (GEANT). The learned policy's advantage under degraded telemetry was small and not firmly established in the packet-level TCP emulation (27 scenarios, Section 6.6): the simulator predicts about 0.008 in max-utilisation, the emulation measured 0.006 with a bootstrap interval excluding zero but a rank test that does not.

8 Limitations and status of claims

- The fluid model has no TCP, queueing or packet dynamics; a 27-scenario TCP emulation (three runs per policy) checks only part of this, and in it the learned policy was only marginally ahead of the heuristic and was the noisiest policy (Section 6.6); the UDP emulation was run once per policy and scenario, and the shadow-mode check covers three scenarios. The guard offers no formal guarantee. It is an empirically evaluated filter, and it misses most harmful proposals under degraded telemetry on Zoo graphs. In the TCP emulation it cost the same mean max-utilisation as in the simulator replay but did not reduce the tail.
- Real traffic is scaled to a target load by factors from 0.02 to 23 (Section 3). Trace capacities for Germany17 and Germany50 are not meaningful in absolute terms, and some traces contain extreme outlier intervals. The conclusion is insensitive to the scaling rule (Section 6.2b), but the absolute numbers are not.
- Telemetry faults are modelled, not measured on a deployed network, and overload far outside the training range hurts the zero-shot policy (Section 6.4).
- We train four policy seeds. The history and age ablations and the DOTE-style and fine-tuned models use one seed. Evaluation uses 5 seeds per Zoo graph and 12 to 16 windows per real-trace cell.
- The heuristic's tuning budget is 80 trials per tuner seed on four synthetic graphs. Tuner-seed spread reaches about 0.02 dScore on Zoo graphs and about 12 points of gain in the delay grid, so single-run comparisons are fragile. We did not test tuning the heuristic on a network's own history, which the fine-tuned learned variants receive.
- Germany50, Brain and the history-based comparison (Table 6) use a single-run heuristic (Appendix A) and were not repeated with the re-tuned heuristic.

- The DOTE-style and fine-tuned models need per-network history, and only Abilene and GEANT have enough. Germany17 and Germany50 have one day of data, and their windows overlap.
- We do not compare with Teal or other published systems: Teal treats staleness only as solver run time and is not a like-for-like baseline for telemetry impairments. Our DOTE-style model is a re-implementation of the idea, not the authors' code.
- We do not claim a general performance win, being first to show degradation under delay (LOGGIA), or production readiness.

9 Conclusion

We asked whether a zero-shot learned split-ratio policy beats a heuristic that is re-tuned for the same telemetry conditions. The answer depends on telemetry quality. On 24 held-out Topology Zoo graphs and replayed SNDlib traces, the re-tuned heuristic matches or beats the learned policy under ideal and rich telemetry, the two tie under SNMP-like telemetry, and the learned policy is ahead under combined delay, noise and drops (+0.033 dScore, 95% interval [+0.018, +0.049]; ahead in all nine real-trace cells). The ranking does not reverse across five tuner seeds, and the learned policy stays ahead when gross errors, lost poll rounds and jitter are added. At packet level the lead shrinks to about 0.006 in maximum utilisation over 27 TCP scenarios, which matches the simulator's prediction but is not established by a rank test. Model-based fluid features, not message passing, carry the result, and the ECMP-fallback guard lowers tail regret in simulation but not in the emulation.

For operators the practical message is to measure telemetry quality and to tune the heuristic for it before adopting a learned policy; the audit tool packages that check. Open work is validating the impairment profiles against production telemetry, measuring latency and recovery time at packet level, running LOGGIA's framework with our baselines, and sweeping noise and drop rate separately.

10 Reproducibility and availability

Code, configuration, trained policies and result tables are in the project repository (https://github.com/Starmarine06/stale-but-safe-te). The environment is Python 3.9 or later (tested with 3.13 on Windows 11 and in the Docker images), installed with `pip install -e ".[dev,rl]"` (torch CPU), and the 43 unit tests run with `python -m pytest -q`; `python experiments/reproduce_tables.py` regenerates nine tables from the committed raw results. The SNDlib matrices are public (Abilene, GEANT, Germany17, Germany50 and Brain) and are not redistributed; they are downloaded and prepared with the repository's `prepare` command. Topology Zoo graphs are not redistributed; a command in the repository downloads them from the public mirror. Each table has one script, and the artefact map in the repository README lists which script and result file produce each table. Seeds, window selection (random 100-interval windows chosen by integer seed), tuner seeds (100 to 104), the model-selection rule and the bootstrap procedures are in the scripts and in Section 5; no selection uses test data. A clean-environment reproduction check (fresh clone, tests, data download and a table slice) is recorded in the repository.

Appendix A: Single-run heuristic results

Our first comparisons used a heuristic tuned once per profile (40 trials, one run). They differ from the re-tuned results in the main text, and the difference is itself a finding. On the 24 Zoo graphs the single-run heuristic's difference to the learned policy (heuristic minus learned dScore, 95% interval over graphs) was -0.016 [-0.030, -0.005] at ideal telemetry and -0.004 [-0.014, +0.004] at rich telemetry, but +0.013 [+0.003, +0.023] under SNMP telemetry and +0.075 [+0.036, +0.120] under degraded telemetry (learned policy wins on 14 graphs, ties 10, loses none; heuristic p99 regret 0.183 against 0.019), more than twice the re-tuned margin of +0.033. On real traces (12 windows per cell, 36 cells) the learned policy appeared ahead in 32 cells, the heuristic ahead in 2 and tied in 2, against 12, 17 and 7 with the re-tuned heuristic (Table 4); the single-run heuristic also fell 3% to 4% below ECMP on Germany17 under degraded telemetry and up to 51% below in individual Abilene windows. Germany50 (50 nodes, larger than any training graph) and Brain were evaluated only with the single-run heuristic: on Germany50 the learned policy gained 33% to 36% against 9% to 31%, and on Brain every method gained about 0.3% in 12

windows. The overload analysis and the emulation also used the single-run heuristic; the qualitative conclusions match the main text.

References

1. Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, A. Tamar. DOTE: Rethinking (Predictive) WAN Traffic Engineering. *NSDI*, 2023, pp. 1557-1581. arXiv:2303.00735.
2. X. Liu, S. Zhao, Y. Cui, X. Wang. FIGRET: Fine-Grained Robustness-Enhanced Traffic Engineering. *ACM SIGCOMM*, 2024. arXiv:2405.04932.
3. M. Ye, J. Zhang, Z. Guo, H. J. Chao. FlexDATE: Flexible and Disturbance-Aware Traffic Engineering with Reinforcement Learning in Software-Defined Networks. *IEEE/ACM Trans. Networking* 31(4), 2023, pp. 1433-1448. doi:10.1109/TNET.2022.3217083.
4. Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, M. Yu. Teal: Learning-Accelerated Optimization of WAN Traffic Engineering. *ACM SIGCOMM*, 2023, pp. 378-393. doi:10.1145/3603269.3604857; arXiv:2210.13763.
5. J. Zhang, M. Ye, Z. Guo, C.-Y. Yen, H. J. Chao. CFR-RL: Traffic Engineering with Reinforcement Learning in SDN. *IEEE JSAC* 38(10), 2020, pp. 2249-2259. doi:10.1109/JSAC.2020.3000371.
6. M. Ye, J. Zhang, Z. Guo, H. J. Chao. Roracle: Enabling Lookahead Routing for Scalable Traffic Engineering with Supervised Learning. *IEEE ICNP*, 2023. doi:10.1109/ICNP59255.2023.10355645.
7. A. Boltres, N. Freymuth, B. Schichtholz, M. Koenig, G. Neumann. Towards Near-Real-Time Telemetry-Aware Routing with Neural Routing Algorithms. *TMLR*, 2026. arXiv:2604.02927.
8. W. Jia, J. Wang, Z. Yan, T. Liu, Y. Ren, K. Lei. EnvelopeTC: Hierarchical Intra-Path Learning for Adaptive Traffic Control. *IEEE JCC*, 2026. doi:10.1109/JCC72984.2026.00071; arXiv:2605.09473.
9. F. Gui, S. Wang, D. Li, L. Chen, K. Gao, C. Min, Y. Wang. RedTE: Mitigating Subsecond Traffic Bursts with Real-time and Distributed Traffic Engineering. *ACM SIGCOMM*, 2024. doi:10.1145/3651890.3672231.
10. H. Mao, M. Schwarzkopf, H. He, M. Alizadeh. Towards Safe Online Reinforcement Learning in Computer Systems ("Training Wheels"). *NeurIPS 2019* (workshop paper; the PDF footer names the 33rd NeurIPS, Vancouver).
11. L. Dinh, P. T. A. Quang, J. Leguay. Safe Load Balancing in Software-Defined-Networking. *Computer Communications*, 2024. arXiv:2410.16846.
12. Y. Xiao, J. Liu, J. Wu, N. Ansari. Leveraging Deep Reinforcement Learning for Traffic Engineering: A Survey. *IEEE Commun. Surveys Tuts.* 23(4), 2021, pp. 2064-2097. doi:10.1109/COMST.2021.3102580.
13. M. Chiesa, G. Retvari, M. Schapira. Lying Your Way to Better Traffic Engineering. *ACM CoNEXT*, 2016. doi:10.1145/2999572.2999585; arXiv:1610.02728.
14. C.-Y. Hong, S. Kandula, R. Mahajan, M. Zhang, V. Gill, M. Nanduri, R. Wattenhofer. Achieving High Utilization with Software-Driven WAN. *ACM SIGCOMM*, 2013, pp. 15–26. doi:10.1145/2486001.2486012.
15. S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu, J. Zolla, U. Hölzle, S. Stuart, A. Vahdat. B4: Experience with a Globally-Deployed Software Defined WAN. *ACM SIGCOMM*, 2013, pp. 3–14. doi:10.1145/2486001.2486019.
16. B. Fortz, M. Thorup. Internet Traffic Engineering by Optimizing OSPF Weights. *IEEE INFOCOM*, 2000, pp. 519–528. doi:10.1109/INFCOM.2000.832225.
17. D. Applegate, E. Cohen. Making Intra-Domain Routing Robust to Changing and Uncertain Traffic Demands: Understanding Fundamental Tradeoffs. *ACM SIGCOMM*, 2003, pp. 313–324. doi:10.1145/863955.863991.
18. A. Valadarsky, M. Schapira, D. Shahaf, A. Tamar. Learning to Route. *ACM HotNets*, 2017, pp. 185–191. doi:10.1145/3152434.3152441.
19. K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, A. Cabellos-Aparicio. Unveiling the Potential of Graph Neural Networks for Network Modeling and Optimization in SDN. *ACM SOSR*, 2019, pp. 140–151. doi:10.1145/3314148.3314357.
20. S. Orlowski, R. Wessaely, M. Pioro, A. Tomaszewski. SNDlib 1.0: Survivable Network Design Library. *Networks* 55(3), 2010, pp. 276-286. doi:10.1002/net.20371 (data source).

21. M. Caria, A. Jukan. On the IP Traffic Matrix Problem in Hybrid SDN/OSPF Networks. arXiv:1610.08256, 2016.
22. M. Roughan. A Case Study of the Accuracy of SNMP Measurements. *J. Electr. Comput. Eng.*, vol. 2010, art. 812979. doi:10.1155/2010/812979.
23. S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, M. Roughan. The Internet Topology Zoo. *IEEE JSAC* 29(9), 2011, pp. 1765-1775. doi:10.1109/JSAC.2011.111002.
24. R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, M. G. Bellemare. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *NeurIPS*, 2021. arXiv:2108.13264.
25. P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger. Deep Reinforcement Learning that Matters. *AAAI*, 2018. arXiv:1709.06560.
26. B. Lantz, B. Heller, N. McKeown. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks. *ACM HotNets*, 2010, pp. 1–6. doi:10.1145/1868447.1868466.
27. N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner. OpenFlow: Enabling Innovation in Campus Networks. *ACM SIGCOMM Computer Communication Review* 38(2), 2008, pp. 69–74. doi:10.1145/1355734.1355746.